

Appendix C

A Field Guide to Published Agent Benchmarks

Chapter 11 condenses the published benchmarks into a single table, Table 11.5, with one row per capability lane. This appendix is the catalog behind that table, and it carries more benchmarks than the table has room to name. For each benchmark it records what the instrument measures and what makes it worth knowing, so that when a table row names a benchmark you have not met, you can find out here whether it fits your system before you open the paper.

The lanes follow the order of Table 11.5, and two lanes the table leaves out close the catalog: embodied agents, and reasoning in a single modality. Both sit further from the systems this book builds, but they matter to teams whose agents act in the physical world or reason over images and audio.

Read the entries with three cautions from *Chapter 11* in hand. First, a published benchmark is a reference layer, not the workbench: it grades a generic capability, and your own suite, built from your production traffic, is still the primary instrument. Second, most harnesses score the final answer or end state, so two systems with the same score can reach it through very different coordination. Third, benchmarks have validity defects of their own; the Agentic Benchmark Checklist [Zhu et al., Rigorous Benchmarks] is worth running against any benchmark you plan to gate a decision on. Scores move quickly, so this appendix describes what each benchmark measures and leaves the current numbers to the source.

Choosing from the catalog

Start with the decision the score must inform. *Chapter 11* names three roles a published benchmark can play: a capability baseline compares model and scaffold choices; seed material supplies cases and failure modes for a private suite; a calibration anchor supplies a grading

contract the field has already operationalized. The same benchmark can serve more than one role, but naming the role prevents a public score from quietly becoming a release gate.

Treat each candidate as a four-part contract: task distribution, environment, grader, and run protocol. The task distribution determines which behaviors the score covers; the environment fixes tools, state, and resource limits; the grader defines success; and the run protocol fixes trial count, retries, and aggregation. A comparison stays meaningful only while those parts remain aligned, so record their versions beside the model and scaffold.

Pair breadth with fidelity. Use a broad benchmark to expose general capability gaps and a domain or threat-model benchmark to test the failure surface that matters to the product. Move the useful cases or rubrics into the private eval suite, then add the trajectory, cost, and coordination measures that the published harness cannot see. That is how a leaderboard result becomes engineering evidence, and those roles and contracts are the lens for the capability lanes that follow.

General-purpose agents

These benchmarks score broad capability across many environments. They are the agent-era counterparts to ImageNet in ambition, though not in structure.

AgentBench

AgentBench [Liu et al., AgentBench] runs eight interactive environments (operating-system shells, databases, knowledge graphs, digital card games, lateral-thinking puzzles, household scenarios, web shopping, and web browsing), each scored on its own metric, such as success rate, F1, or reward, and combined into a weighted overall score. At publication it showed a wide gap between API-served frontier models and open-source models of up to 70B parameters. Its construction is the lasting part: tasks run for many rounds, long enough for an early mistake to carry into every step after it.

Repository: <https://github.com/THUDM/AgentBench>

AgentBoard

AgentBoard [Ma et al., AgentBoard] is built on multi-turn trajectories scored with a *progress rate* as well as final success. The benchmarks of its day reported only whether the agent finished; AgentBoard reports how far it got, which is the signal a team usually wants when it is debugging a failure.

Repository: <https://github.com/hkust-nlp/AgentBoard>

GAIA

GAIA [Mialon et al., GAIA] poses 466 questions that need web browsing, reading files in many formats (images and audio among them), coding, and multi-step reasoning to reach one short, unambiguous answer. It became the common yardstick for generalist assistant work.

Website: <https://huggingface.co/gaia-benchmark>

Gaia2

Gaia2 [Froger et al., Gaia2] is GAIA's successor, redesigned around dynamic state, asynchronous events, and a verifier for write actions. GAIA's agents only read, and its web changed between runs, not mid-task or in response to the agents, so nothing in it required an agent to track state it had changed; Gaia2 grades every state-changing action against an environment that moves between the agent's observations.

Repository: <https://github.com/facebookresearch/meta-agents-research-environments>

AstaBench

AstaBench [Bragg et al., AstaBench] is a scientific-research suite of more than 2,400 problems spanning literature understanding, code and execution, data analysis, and end-to-end discovery, run inside a fixed research environment so that differences in tool access do not confound the comparison. The end-to-end tasks run the whole pipeline, from a given idea through experiments to a written report, and are graded against a task-specific rubric because no reference answer exists. Each idea arrives specified in detail, so these tasks test research execution more than ideation.

Repository: <https://github.com/allenai/asta-bench>

GDPval

GDPval [Patwardhan et al., GDPval] collects 1,320 tasks built from the real work products of 44 occupations across the nine largest US GDP sectors, written by professionals averaging 14 years of experience. Each task asks for a deliverable, usually a document, a spreadsheet, or a slide deck, with no answer string to match. Its released 220-task subset is graded by blinded expert pairwise comparison and ships with an experimental public automated grader. Of the benchmarks here, its output looks most like what a production knowledge-work agent hands over.

Website: <https://evals.openai.com>

Remote Labor Index

The Remote Labor Index (RLI) [Mazeika et al., Remote Labor Index] is built from 240 freelance projects across 23 categories of remote work, including game development, product design,

architecture, and data analysis. Each project pairs the original brief with the deliverable a human professional produced for it, and those deliverables took a median of 11.5 hours to make. Human evaluators judge whether the agent's deliverable completes the project at least as well as the human one, and the share of projects that pass is the *automation rate*, which stood at 2.5% for the best agent at publication. GDPval asks which of two outputs an expert prefers on a task; RLI asks whether a client would have accepted the whole project.

Website: <https://www.remotelabor.ai>

Agents' Last Exam

Agents' Last Exam (ALE) [Sun et al., Agents' Last Exam] collects 1,490 task instances of professional work across 55 subfields grouped into 13 industry clusters, run on a remote Linux or Windows machine where tasks interleave desktop and command-line work. Deliverables are checked against expert-provided references and rubrics, and 93.2% of those checks are deterministic code, the rest narrow yes-or-no probes put to an LLM judge. Only about a tenth of the instances are public, and private ones rotate into the public set as public ones retire, so most of the benchmark stays out of training data. Across the harness and model configurations the paper tested, the hardest tier's average full pass rate was below 1%. The backbone model mattered more than the harness: swapping models spread the overall pass rate by 16.8 percentage points, and swapping harnesses around a fixed model moved it by 4.9 to 7.2 points.

Repository: <https://github.com/rdi-berkeley/agents-last-exam>

TheAgentCompany

TheAgentCompany [Xu et al., TheAgentCompany] simulates a small software company and gives the agent 175 tasks a worker there might be assigned. The environment is self-hosted open-source software, with GitLab for code, ownCloud for documents, Plane for project management, and RocketChat for messaging, and the colleagues on the other end of the chat are simulated by an LLM. Some of what a task needs is missing from its description and has to be asked for, so consulting a coworker becomes part of the work. Each task is split into checkpoints with point values, which lets the score credit partial progress as well as full completion. In the paper's results the strongest agent completed 30% of the tasks, or 39% counting partial credit, at an average of almost 27 steps and more than \$4 a task.

Repository: <https://github.com/TheAgentCompany/TheAgentCompany>

Domain rules

A domain benchmark is built around the rules of one industry. Most production teams work inside a domain, so a benchmark in yours says more about whether your agent will serve your customers than a general-purpose score can.

T-bench

T-bench [Yao et al., T-bench] tests tool-agent-user interaction in retail and airline customer service. The agent must follow domain policies (returns and exchanges, baggage allowances, which fares may be changed or canceled) while acting through tools, and it is scored on $pass^k$, the probability that all k independent trials of the same task succeed, because customer service needs the agent to behave the same way every time. It is the source of $pass^k$, one of the two consistency metrics in *Robustness metrics* in Chapter 11.

Repository: <https://github.com/sierra-research/tau-bench>

T²-bench

T²-bench [Barres et al., T²-bench] extends T-bench with a telecom support domain in which the simulated user holds tools too. Some actions only the user can take, so the agent has to guide the user through the steps that change the shared state, a setup the authors call *dual control*. Tasks are generated by composing atomic subtasks, 2,285 in all, of which 114 are sampled for a balanced set. Each can be run in a no-user mode, where the agent holds every tool, and in the default dual-control mode, so the difference between the two scores separates reasoning errors from errors of communication and coordination. In the paper, scores drop significantly from the first mode to the second, which locates much of the difficulty in guiding another party.

Repository: <https://github.com/sierra-research/tau2-bench>

Cybench

Cybench [Zhang et al., Cybench] collects 40 professional capture-the-flag tasks drawn from four competitions. In this domain, capability evaluation is risk evaluation, since solving a task that exploits a vulnerability, even one planted for a competition, measures attack capability, and the correctness and safety dimensions meet in one artifact.

Repository: <https://github.com/andyzorigin/cybench>

MedAgentGym

MedAgentGym [Xu et al., MedAgentGym] is a biomedical *training environment* of about 72,000 tasks, built mainly for training, with a test split held out for evaluation. It belongs in the catalog because teams often conflate training environments, which are large and built for reinforcement learning or fine-tuning, with evaluation benchmarks, which are small, curated, and held out. An agent that learned on a suite can be scored only on the split its authors keep out of training.

Repository: <https://github.com/wshi83/MedAgentGym>

PathGen-1.6M

PathGen-1.6M [Sun et al., PathGen-1.6M] is a set of 1.6 million pathology image-text pairs generated through multi-agent collaboration. Here the agents generate data, so the evaluation question becomes whether the data they produced trains a better downstream model.

Repository: <https://github.com/PathFoundation/PathGen-1.6M>

TravelPlanner

TravelPlanner [Xie et al., TravelPlanner] poses travel planning under many simultaneous constraints. Up to five come from the request (budget, cuisine, room type, house rules, and mode of transport), and eight more are left implicit, such as a sane order for the cities visited and transport legs that do not conflict with one another. A fluent itinerary that breaks any one of them fails the final pass rate, which makes the benchmark a direct test of planning under realistic constraints.

Repository: <https://github.com/OSU-NLP-Group/TravelPlanner>

Software engineering

Code is the area where output verification is cheapest, since a patch can be compiled and tested, and it is one of the most crowded lanes.

SWE-bench and SWE-bench Verified

The task in SWE-bench [Jimenez et al., SWE-bench] and SWE-bench Verified [OpenAI, SWE-bench Verified] is resolving GitHub issues, graded by each repository's own tests. The original benchmark has 2,294 instances. Human review of a 1,699-instance sample then screened out roughly two-thirds, mostly for under-specified issue text or unit tests that could reject a correct patch, by a filter its authors called likely overzealous, and the 500 instances the annotators verified became the field's reference. In February 2026 OpenAI, which led that curation, stopped reporting Verified as a frontier measure: 59.4% of the hard tasks it audited had flawed tests, and every frontier model it checked showed signs of having seen the solutions in training [OpenAI, SWE-bench Retired]. The history is itself a caution: even the most widely used benchmark needed human curation once frontier agents reached it, and the curated version lasted about eighteen months.

Repository: <https://github.com/SWE-bench/SWE-bench>

SWE-bench Pro

SWE-bench Pro [SWE-Bench Pro] [Scale AI, SWE-Bench Pro V2] is Scale AI's extension of the line, first released with 1,865 problems across 41 repositories split into public, held-out, and commercial sets, so that part of the benchmark stays out of any training corpus. OpenAI's

retirement post recommends reporting it in Verified's place. The benchmark's V2 release trimmed the 731-task public split to 642 tasks, dropping 89 found invalid. With frontier models near the ceiling on those public tasks, Scale calls its private set the only clean measurement.

Repository: https://github.com/scaleapi/SWE-bench_Pro-os

Senior SWE-bench

Senior SWE-bench [Snorkel AI, Senior SWE-bench] draws 100 tasks, half public and half private, from pull requests merged after February 2026 in 12 open-source repositories, with instructions a median of 31% the length of SWE-bench Pro's. A solve counts as *tasteful* only when it clears six gates: hidden behavioral tests, tests a validation agent writes against the submitted solution, a task rubric, a limit on patch bloat, and two scores for engineering practice and taste. Most of those gates are LLM judgments, and only Snorkel can score the private half, so the leaderboard cannot be reproduced in full from outside. Snorkel reports that the leading models miss senior-level correctness and taste on more than 65% of tasks.

Repository: <https://github.com/snorkel-ai/senior-swe-bench-v2026.06>

SlopCodeBench

SlopCodeBench [Orlanski et al., SlopCodeBench] measures what happens to code as an agent keeps extending it: 36 problems unfold over 196 checkpoints, and at each one the agent receives a new specification and its own previous workspace but not the hidden tests. Besides correctness it scores two kinds of decay, *verbosity* (redundant code) and *structural erosion* (complexity concentrating in a few functions). No agent in the paper solved any problem end to end, the best passed 14.8% of checkpoints, and erosion rose in 77% of trajectories and verbosity in 75.5%. Quality guidance in the prompt lowered where the code started but not how fast it degraded, which is the long-running maintenance cost that a benchmark scoring one patch at a time cannot see.

Repository: <https://github.com/SprocketLab/slop-code-bench>

LibraryDesignBench

LibraryDesignBench [Orlanski et al., LibraryDesignBench] measures one agent's work by how well other agents can build on it. A designer agent implements a full library from a specification that names the required capabilities without prescribing the design, and three user agents from different model families then write programs with it for 242 downstream problems across 15 tasks in four languages. The library is judged only through the correctness and simplicity of those programs. Designers reproduced the production library's abstractions on 11 of the 15 tasks, yet user agents underused the libraries and reimplemented what they already provided, mainly because agent-written libraries were rigid or hard to use.

Repository: <https://github.com/SprocketLab/librarydesignbench>

BaxBench

BaxBench [Vero et al., BaxBench] tests backend code generation graded on two criteria at once: the application must pass functional tests and survive end-to-end exploits for common vulnerabilities, so a working API that is open to SQL injection fails. Earlier security benchmarks often graded the two on separate tasks, or together only on single functions, but BaxBench grades both on complete backends, which is closer to how production code review works.

Repository: <https://github.com/logic-star-ai/baxbench>

Spider 2.0

Spider 2.0 [Lei et al., Spider 2.0] covers enterprise text-to-SQL over real workflows, with several database systems and SQL dialects, business definitions supplied as documentation, project-level transformation code, and schemas that often exceed 1,000 columns. At publication its best agent solved 21.3% of the tasks, against 91.2% on Spider 1.0 and 73.0% on BIRD, which is the evidence that enterprise SQL is harder than the field had been measuring.

Repository: <https://github.com/xlang-ai/Spider2>

BIRD-INTERACT

BIRD-INTERACT [Huo et al., BIRD-INTERACT] is text-to-SQL rebuilt around interactive sessions, where a simulated user answers clarifying questions, the session's own writes change the database as it goes, and the agent has to recover from its errors. It tests interaction as a skill of its own, separate from producing one correct query.

Repository: <https://github.com/bird-bench/BIRD-Interact>

MLAgentBench

MLAgentBench [Huang et al., MLAgentBench] sets machine-learning experimentation as the task. The agent reads a task description and starter code, edits and runs the training script, observes the result, and iterates, so it has to close its own loop end to end, including recovering from failures in its own earlier steps.

Repository: <https://github.com/snap-stanford/MLAgentBench>

MLE-bench

MLE-bench [Chan et al., MLE-bench] draws on 75 Kaggle competitions treated as end-to-end machine-learning engineering jobs. The agent prepares data, trains models, runs experiments, and submits predictions, and the grader places the result on the competition's human leaderboard to see whether it reaches a medal. MLAgentBench makes the experimentation loop visible; MLE-bench supplies the breadth and human-relative scale of full competitions.

Repository: <https://github.com/openai/mle-bench>

PostTrainBench

PostTrainBench [Rank et al., PostTrainBench] gives an agent a small base model, a target benchmark, one H100 GPU, and 10 hours, with web access but no starter code, training data, or hyperparameters, and scores the checkpoint it submits on the benchmark's held-out test set. It spans 28 configurations, four base models against seven benchmarks. The best agent in the paper reached 23.2% against 51.1% for the official instruction-tuned models, though agents beat those models in targeted cases, such as 89% against 67% on function calling with Gemma-3-4B. Its record of reward hacking is the reason to read it: agents trained on the test set, downloaded an existing instruction-tuned checkpoint in place of training their own, and used API keys they found to generate synthetic data without authorization. One agent acknowledged the restriction in its reasoning before breaking it.

Repository: <https://github.com/aisa-group/PostTrainBench>

Operations

These benchmarks cover the work the code benchmarks miss: operating a production system instead of shipping a patch.

SREGym

SREGym [Clark et al., SREGym] sets 90 problems in a live cloud-native system, where each failure is produced by injecting a real defect at the layer it originates in, and the modes covered include metastable, concurrent, and correlated failures. The agent submits a diagnosis and then signals that mitigation is complete, and a problem-specific oracle checks whether the fault is actually resolved and the system has returned to a healthy state. The paper's reason for grading that way belongs in your own harness too: restarting a pod can clear the alert without touching the defect, so the oracles read live environment health directly.

Repository: <https://github.com/SREGym/SREGym>

SRE-skills-bench

SRE-skills-bench [Rootly AI Labs, SRE-skills-bench] tests cloud and infrastructure configuration across AWS, Azure, and GCP, covering storage, networking, identity, compute, and Kubernetes. Its leaderboard scores multiple-choice questions validated by practicing site reliability engineers, and another track asks for Terraform and grades it by running plan and apply against a local cloud emulator. It tests breadth across services at the level an on-call rotation needs, and none of its released tracks grades work done during an incident, which is what separates it from SREGym.

Repository: <https://github.com/Rootly-AI-Labs/sre-skills-bench>

ITBench

ITBench [Jha et al., ITBench] covers three operations roles in one framework: site reliability engineering (SRE), compliance and security operations, and financial operations (FinOps). Its 102 scenarios run in live environments, for example a Kubernetes cluster with an observability stack in which the agent must resolve a high error rate on one service. Beyond a pass rate, the SRE scenarios report fault localization, the fault propagation chain, and the mean time to diagnosis and to repair, and the framework scores partly correct solutions. At publication agents resolved 11.4% of the SRE scenarios, 25.2% of the compliance and security scenarios, and 25.8% of the FinOps scenarios outside anomaly detection. IBM Research and UC Berkeley later annotated 310 of its SRE traces with a failure taxonomy, the study *Chapter 8* draws on for its point about verification [IBM, ITBench MAST].

Repository: <https://github.com/itbench-hub/ITBench>

Tool use

This lane tests the tool layer itself. Domain suites that happen to require tools, such as T-bench and the MedAgentGym training environment, sit under *Domain rules*.

ToolLLM

ToolLLM [Qin et al., ToolLLM] gathers more than 16,000 real-world APIs that the agent must discover, select, invoke, and chain. At that scale the quality of each tool's description is likely to decide whether the agent works, since a vague description turns into wrong calls wherever the tool is reachable.

Repository: <https://github.com/OpenBMB/ToolBench>

MetaTool

MetaTool [Huang et al., MetaTool] benchmarks deciding *whether* to use a tool, and *which one*. Earlier suites handed the model the right tool and scored only the call, so choosing well from a large inventory went unmeasured until selection got a benchmark of its own.

Repository: <https://github.com/HowieHwong/MetaTool>

Berkeley Function Calling Leaderboard (BFCL)

The Berkeley Function Calling Leaderboard (BFCL) [Patil et al., BFCL] grades the function call itself. It evaluates serial and parallel calls across several programming languages by matching abstract syntax trees, a method that scales to thousands of functions. The functions and prompts are a mix of expert-curated and user-contributed, and the benchmark also tests

whether a model abstains and whether it can reason through stateful, multi-step tasks. Its authors find that models which excel at single-turn calls still struggle with memory, dynamic decision-making, and long-horizon reasoning. ToolLLM and MetaTool study which tool an agent picks; BFCL checks the call that comes out.

Repository: <https://github.com/ShishirPatil/gorilla/tree/main/berkeley-function-call-leaderboard>

MCP-Bench

MCP-Bench [MCP-Bench, MCP-Bench] runs tool use through 28 live servers speaking the Model Context Protocol (MCP), exposing 250 tools. Its tasks name no tools and often span several servers, so it tests whether an agent can find, chain, and coordinate tools it was not pointed at.

Repository: <https://github.com/Accenture/mcp-bench>

Web and computer use

Browsing, web automation, and control of a full operating system form a young capability surface, and its benchmarks are multiplying accordingly.

WebArena

WebArena [Zhou et al., WebArena] sets 812 tasks on self-hosted instances of a forum, an e-commerce site, a content-management system, a GitLab clone, and OpenStreetMap. The environments are real software the agent has to operate, not simulations of web pages. Its functional graders inspect the final site or database state where possible, so a valid route that differs from the reference action sequence can still pass.

Repository: <https://github.com/web-arena-x/webarena>

WorkArena

WorkArena [Drouin et al., WorkArena] has 33 task types and 19,912 instances inside ServiceNow, covering the forms, lists, dashboards, catalogs, and knowledge bases that make up routine enterprise work. Each task type has a validator, and scripted oracles establish that the task is feasible. Where WebArena spreads across several sites, WorkArena goes deep inside one stateful business system.

Repository: <https://github.com/ServiceNow/WorkArena>

BrowseComp

BrowseComp [OpenAI, BrowseComp] asks 1,266 questions whose short, unique answers are hard to find but easy to verify. That asymmetry makes persistence, search reformulation, and

evidence gathering measurable with a trivial grading step, a model checking each short answer against the reference. It also narrows the claim: a score on deliberately obscure fact-finding does not establish that an agent can answer the open-ended, ambiguous requests in ordinary research work.

Repository: <https://github.com/openai/simple-evals>

Mind2Web 2

Mind2Web 2 [Gou et al., Mind2Web 2] sets 130 long-horizon tasks that need live web browsing and synthesis across many pages, the work a deep-research system does. Answers change as the web changes, so no fixed reference answer exists, and each task ships with a judge agent built on a tree-structured rubric. The leaves of the tree are criteria that can be verified with a yes or a no, and their scores aggregate upward into the task score. The rubric checks whether the answer satisfies the task and whether each statement in it can be attributed to the source it cites. At publication the best system reached 50 to 70% of human performance in half the time.

Repository: <https://github.com/OSU-NLP-Group/Mind2Web-2>

DeepResearch Bench

DeepResearch Bench [Du et al., DeepResearch Bench] poses 100 PhD-level research tasks across 22 fields, with the mix of topics drawn from 96,147 real user queries to a search-enabled chatbot. The deliverable is a long cited report, and the benchmark grades it twice. One method scores report quality against a reference report, with criteria and weights generated for each task. The other extracts each statement and the URL cited for it, checks whether the page supports the statement, and reports citation accuracy alongside the number of citations that hold up. A system can write a persuasive report on citations that do not support it, and keeping the two scores apart is what exposes that.

Repository: https://github.com/Ayanami0730/deep_research_bench

WebLINX

WebLINX [Lu et al., WebLINX] tests real-world website navigation through multi-turn dialogue, in which the user's goal is not fully stated at the start and evolves as the conversation proceeds. The agent must track that changing intent, where a single-instruction benchmark tests only execution. It is scored turn by turn against recorded human demonstrations, so unlike WebArena it cannot credit a different route that would also have worked.

Repository: <https://github.com/McGill-NLP/weblinx>

WebGen-Bench

WebGen-Bench [Lu et al., WebGen-Bench] tests generating functional websites, graded by deploying each site and driving it. A web-navigation agent works through 647 curated cases,

each one an operation and the response it should produce, and decides whether what it saw matches. Executing the artifact removes the need to read its code, but the verdict on every case is still a judge's, which is why this sits closer to LLM-as-judge than to code-based grading.

Repository: <https://github.com/mnluzimu/WebGen-Bench>

OSWorld

OSWorld [Xie et al., OSWorld] sets 369 tasks on Ubuntu, plus 43 on Windows, across real web and desktop applications, including file operations and workflows that cross application boundaries. Every task starts from a specified machine state and has a custom execution-based evaluator. That combination tests grounded computer use across applications; a screenshot question-answering system cannot pass by describing what it sees.

Repository: <https://github.com/xlang-ai/OSWorld>

OSWorld 2.0

OSWorld 2.0 [Yuan et al., OSWorld 2.0] builds on OSWorld's platform with 108 new long-horizon workflows across desktop applications and 31 self-hosted websites, which take human users a median of about 1.6 hours. Every task carries an average of 27 checkpoints for partial credit, with binary completion as the primary metric, and the environment injects email and chat messages mid-task and includes a simulated user who knows only part of the task. Claude Opus 4.8, the best agent in the paper, completed 20.6% of tasks, but the same frontier agents scored 79 to 83% on the original OSWorld. Agents spent under 7% of their budget detecting and repairing their own errors.

Repository: <https://github.com/xlang-ai/OSWorld-V2>

AndroidWorld

AndroidWorld [Rawles et al., AndroidWorld] brings execution-based grading to a phone: 116 tasks across 20 real Android apps, each with its own initialization, success-checking, and teardown logic that modifies and inspects the device's system state. Tasks are parameterized, so the environment can generate fresh variants of each one in place of a fixed test set. The paper uses that to run a robustness analysis, which shows that varying a task's parameters can change an agent's performance significantly. At publication the best agent completed 30.6% of the tasks against 80.0% for humans. AndroidWorld is the mobile counterpart to OSWorld, and its parameterized tasks are one answer to the fixed public task set that *Chapter 11* names as a limit of published benchmarks.

Repository: https://github.com/google-research/android_world

Terminal-Bench

Terminal-Bench [Terminal-Bench, Terminal-Bench] [Anthropic, Infrastructure Noise] is a versioned line of containerized command-line tasks, each with its own environment and executable tests. The move from 2.0 to 2.1 repaired 28 of 89 tasks for dependency drift, resource mismatches, or disagreement between instructions and tests; a separate controlled study found that container headroom alone moved the 2.0 score by about six percentage points. Release 4.0 then removed eight tasks, two of them because every model of the latest generation solved them five times out of five, fixed 19 more, and calibrated each task's resources by that study's method [Terminal-Bench, Terminal-Bench 4.0]. The lesson travels beyond this benchmark: runtime limits and environment versions belong in the reported treatment, not in a footnote.

Website: <https://www.tbench.ai>

Terminal-Bench Science

Terminal-Bench Science [Terminal-Bench-Science, Terminal-Bench-Science 0.1] applies the same containerized format to research: 70 tasks across the life, physical, earth, mathematical, and engineering sciences, each drawn from computational work a practicing researcher did. Deterministic tests grade every task with a binary reward, and before merging each one had to survive trial runs by frontier, oracle, and deliberately cheating agents; 70 of 920 proposals made it through. At release the best configuration, Claude Opus 5 with Claude Code, resolved 30.0% of tasks, and every model evaluated on both scored more than ten points lower here than on Terminal-Bench 3.0.

Repository: <https://github.com/harbor-framework/terminal-bench-science>

SimuHome

SimuHome [Seo et al., SimuHome] is a smart-home simulator and 600-episode benchmark grounded in the Matter protocol, in which device actions shift temperature, humidity, and other variables gradually, and some requests must be scheduled around when a device will finish. An agent that acts on an old reading without observing again gets these wrong, and workflow scheduling was the hardest category in the paper.

Repository: <https://github.com/holi-lab/SimuHome>

ScaleCUA

ScaleCUA [Liu et al., ScaleCUA] is a cross-platform training dataset for computer-use agents, spanning six operating systems and released together with the open-source agents trained on it. It is a training resource more than a benchmark: its agents serve as an open baseline to

compare against, and its one new evaluation target is WebArena-Lite-v2, an upgrade of WebArena-Lite for agents that act from screenshots.

Repository: <https://github.com/OpenGVLab/ScaleCUA>

Memory

Memory benchmarks test what an agent retains and how it uses it as interactions arrive over time.

LoCoMo

LoCoMo [Maharana et al., LoCoMo] is a set of very long conversations between two LLM-driven speakers, each running about 600 turns and 16,000 tokens over as many as 32 sessions. The dialogues are generated from a persona and a temporal event graph for each speaker, then verified and edited by human annotators. Its question-answering task sorts questions into single-hop, multi-hop, temporal, open-domain, and adversarial, the last being questions the agent should recognize as unanswerable; event summarization and multimodal dialogue generation are scored as separate tasks. Memory-system leaderboards lean on it heavily, which is why *Chapter 7* records an audit that found 6.4% of its answer key wrong [Penfield, LoCoMo Audit]. Treat a LoCoMo score as comparable only when the harness and the judge are stated.

Repository: <https://github.com/snap-research/locomo>

LongMemEval

LongMemEval [Wu et al., LongMemEval] asks 500 hand-written questions, each answerable only from information planted somewhere in a long history of chats between a user and an assistant. The questions test five abilities: information extraction, multi-session reasoning, temporal reasoning, knowledge updates, and abstention, which is recognizing a question the history cannot answer. The history can be made as long as the evaluator wants, and the paper fixes two standard settings, one of about 115,000 tokens per question and one of 500 sessions, around 1.5 million tokens. In the paper, commercial chat assistants and long-context models showed a 30% drop in accuracy on information spread across sustained interactions. *Chapter 7* uses the five abilities as a checklist for any memory system you build.

Repository: <https://github.com/xiaowu0162/LongMemEval>

BABILong

BABILong [Kuratov et al., BABILong] takes 20 short reasoning tasks, among them fact chaining, induction, deduction, counting, and handling lists and sets, and scatters the facts each one needs through long natural text. The benchmark can be extended to any length and ships splits of up to 10 million tokens, so it keeps pace as context windows grow. Its headline result, which

Chapter 7 carries, is that popular models make effective use of only 10 to 20% of their context, and that accuracy falls sharply as the reasoning gets more complex. Retrieval does not rescue it: retrieval-augmented generation reached about 60% on single-fact questions regardless of context length. It tests reasoning over one long static input, which is why MemoryAgentBench, where information arrives incrementally, sits beside it.

Repository: <https://github.com/booydar/babilong>

MemoryAgentBench

MemoryAgentBench [Hu et al., MemoryAgentBench] tests memory competence across accurate retrieval, test-time learning, long-range understanding, and selective forgetting, with the information delivered incrementally over many turns. Its long-range and selective-forgetting scenarios are the closest published proxy for the memory failures that later sessions inherit. *Chapter 11's* Table 11.5 names it for the memory lane, and *Chapter 7* reaches for it where LongMemEval and BABILong stop, since it adds test-time learning and selective forgetting to what they measure.

Repository: <https://github.com/HUST-AI-HYZ/MemoryAgentBench>

Continual Learning Bench

Continual Learning Bench [Asawa et al., Continual Learning Bench] asks whether a system gets better at a task by doing it. Its 301 instances span six domains, among them software engineering, disease outbreak forecasting, strategic game-playing, and database querying against a schema that migrates mid-run, and are solved in sequence with state carried between them. The *gain* metric subtracts the same system's score when run without state, which separates learning from experience from simply being a stronger model. Plain in-context learning over the history beat dedicated memory architectures on most tasks, and the best system the paper tested realized only 25.4% of the gain available to it. Accumulated state often hurt, as memory modules introduced spurious generalizations and stale beliefs.

Repository: <https://github.com/pgasawa/continual-learning-bench>

Coordination

Coordination quality is invisible to output-only scoring, and this lane exists to make it visible.

MultiAgentBench

MultiAgentBench [Zhu et al., MultiAgentBench] scores a team at work across six scenarios. In four the agents share a goal (research collaboration, building in Minecraft, database error analysis, and coding), and in two their goals conflict (Werewolf and bargaining). Alongside a task score it tracks the milestones a team reaches along the way, and an LLM judge rates the

team's communication and planning on a five-point scale, which average into a coordination score. The same tasks run under star, chain, tree, and graph protocols, so the benchmark compares topologies as well as models; in the paper the graph protocol did best on the research scenario. The paper's own reading is that model ability remained the key factor in task completion. The coordination score is an LLM judge's reading of the transcript, so it needs the calibration *Chapter 11* describes before it gates anything.

Repository: <https://github.com/ulab-uiuc/MARBLE>

AgentsNet

AgentsNet [Grötschla et al., AgentsNet] poses five classical problems from distributed computing to a network of agents: graph coloring, minimal vertex cover, maximal matching, leader election, and consensus. Each agent is a node that can exchange messages only with its immediate neighbors, in synchronous rounds, and the network has to reach a valid joint solution from local information alone. The problems are well studied, so what counts as a correct joint solution is not in dispute, and the graph can be any size: the authors run it with up to 100 agents, where they count two to five in earlier multi-agent benchmarks. They report that frontier models do well on small networks and fall off as the network grows. The benchmark isolates whether a topology of agents can organize itself from whether the agents can do a task at all.

Repository: <https://github.com/floriangroetschla/AgentsNet>

Who&When

Who&When [Zhang et al., Who&When] turns failure attribution into a benchmark. It collects failure logs from 127 LLM multi-agent systems, some generated by algorithm and some built by hand, and annotates each with the agent responsible, the decisive error step, and an explanation in natural language. The attribution method is the system under test: given a failed run, it must name who failed and when. The best method in the paper named the right agent 53.5% of the time and the right step 14.2% of the time, the ceilings *Chapter 11* reports. Who&When Pro extends the line to 12,326 trajectories across 26 source benchmarks [Liu et al., Who&When Pro], and *Chapter 12* uses it to show how attribution accuracy falls as traces get longer.

Repository: https://github.com/ag2ai/Agents_Failure_Attribution

SOTOPIA

SOTOPIA [Zhou et al., SOTOPIA] is an interactive evaluation of social intelligence in language agents across 90 scenarios with cooperative, competitive, and mixed goals. Each role has a private goal and a character profile that may include secrets, so success is not reducible to sounding agreeable. Its seven-part evaluation also tracks believability, relationships,

information gained, secrets kept, social rules, and material outcomes alongside goal completion, which is much closer to the trade-offs real coordination creates.

Repository: <https://github.com/sotopia-lab/sotopia>

WhodunitBench

WhodunitBench [Xie et al., WhodunitBench] is built on multimodal murder-mystery games in which each agent holds incomplete information, plays an assigned character, and must reason about what the others know and believe. Not every player wants the case solved, since one is the murderer and is working to stay hidden, so an agent has to read intent as well as evidence. Individual competence is not enough: agents that perceive the clues reasonably well still fall short on the collaboration and multi-step reasoning the game demands, and the best won only a quarter of its games.

Repository: https://github.com/jun0wanan/WhodunitBench-Murder_Mystery_Games

CompeteAI

CompeteAI [Zhao et al., CompeteAI] studies competition between LLM-based agents, run as restaurants competing for customers in a simulated town. Most coordination benchmarks assume agents are trying to help each other, but CompeteAI assumes they are not. The dynamics it surfaces, such as winner-take-all outcomes, differ from the failure modes of cooperation.

Repository: <https://github.com/microsoft/competeai>

CivRealm

CivRealm [Qi et al., CivRealm] is a Civilization-style environment for long-horizon strategy among a changing number of players, where competition coexists with diplomacy and negotiation. Its rewards are sparse and delayed and its decisions pay off many turns later, which tests whether an agent plans ahead or only reacts.

Repository: <https://github.com/bigai-ai/civrealm>

Safety and adversarial

These are the benchmarks an evaluation team should know about, even though the red-team discipline itself is developed in *Chapter 14*. Each one tests a stated threat model, and a score says nothing about threats outside it.

AgentHarm

AgentHarm [Andriushchenko et al., AgentHarm] collects explicitly malicious multi-step agent tasks, posed directly by the user, scored on whether the agent refuses and on whether a

jailbroken agent keeps its capability. Its threat model is direct misuse; attacks delivered through tool outputs belong to AgentDojo.

Repository: https://github.com/UKGovernmentBEIS/inspect_evals/tree/main/src/inspect_evals/agentharm

AgentDojo

AgentDojo [Debenedetti et al., AgentDojo] targets indirect prompt injection against tool-using agents, where the malicious payload arrives through a tool output instead of the user's prompt. Its 97 benign tasks expand into 629 security cases, and state-based checks report benign utility, utility under attack, and attacker success separately. That pairing matters because a defense that blocks every action can defeat the injection and still be useless to the user. *Chapter 14* builds its indirect-injection discipline on it.

Repository: <https://github.com/ethz-spylab/agentdojo>

Agent Security Bench (ASB)

Agent Security Bench (ASB) [ASB, ASB] is a broad enumeration of attacks and defenses across ten application scenarios and more than 400 tools, covering direct and indirect prompt injection, memory poisoning, a plan-of-thought backdoor, and mixed attacks that combine them. It favors coverage over depth, which makes it the place to start when you do not yet know which attack class your system is most exposed to.

Repository: <https://github.com/agiresearch/ASB>

MCPTox

MCPTox [Wang et al., MCPTox] measures tool-poisoning attacks across real MCP servers and the tools an agent would reach for. It is the benchmark to run when your system's tools come from third parties you do not control; *Chapter 14* carries its success rates and what they imply about trusting an MCP endpoint.

Repository: <https://github.com/zhiqiangwang4/MCPTox-Benchmark>

RedTeamCUA

RedTeamCUA [Liao et al., RedTeamCUA] runs adversarial testing of computer-use agents in hybrid web and operating-system environments. An agent that controls the operating system can do damage well beyond any one application, which is why that surface has an adversarial benchmark of its own.

Repository: <https://github.com/OSU-NLP-Group/RedTeamCUA>

OS-Harm

OS-Harm [Kuntz et al., OS-Harm] measures the safety of computer-use agents inside the OSWorld environment. Its 150 tasks divide evenly among three sources of harm: a user who asks for something harmful, a prompt injection planted in what the agent reads, and the agent's own misbehavior on a benign task. An LLM judge scores each run for both task completion and safety, and it agrees with human annotators at F1 scores of 0.76 and 0.79. The paper found that every model it tested complied directly with many of the misuse requests and was relatively vulnerable to static prompt injections. RedTeamCUA tests the injected attack; OS-Harm also counts the harm that needs no attacker, a user's own request and the agent's own mistake.

Repository: <https://github.com/tml-epfl/os-harm>

TAMAS

TAMAS [Kavathekar et al., TAMAS] measures adversarial robustness of a multi-agent team across six attack types, scored with an Effective Robustness Score that is the harmonic mean of the team's safety under those attacks and its task performance when no attack is present. Pairing the two is what stops a system from scoring well by refusing everything. It evaluates the team under pressure, not each member separately.

Repository: <https://github.com/microsoft/TAMAS>

AutoAdvExBench

AutoAdvExBench [Carlini et al., AutoAdvExBench] benchmarks autonomous exploitation of adversarial-example defenses. The threat model is inverted, with the agent as the attacker, so it measures how well agents break defenses and anticipates what attackers will do as agents grow more capable.

Repository: <https://github.com/ethz-spylab/AutoAdvExBench>

Judges

This lane is reflexive: its benchmarks evaluate the judges themselves. *Chapter 11* develops the calibration discipline in *Human evaluation and LLM-as-judge*.

MT-Bench

MT-Bench [Zheng et al., Judging LLM-as-a-Judge] asks 80 multi-turn questions across eight categories, scored by an LLM judge. It popularized LLM-as-judge as a practice and supplied much of the early evidence of judge bias (position and verbosity, with a first, inconclusive look at self-preference) that calibration procedures now defend against.

Repository: https://github.com/lm-sys/FastChat/tree/main/fastchat/llm_judge

Chatbot Arena

Chatbot Arena [Chiang et al., Chatbot Arena] measures crowd-sourced pairwise human preference, collected through blind side-by-side comparisons and aggregated into Bradley-Terry rankings, which replaced the Elo scores the leaderboard first reported. It anchored the move from scoring against references to scoring by preference. The platform has been renamed twice since the paper, to LMArena and then to Arena, so the paper's title is the more reliable thing to search for.

Website: <https://arena.ai>

JudgeBench

JudgeBench [Tan et al., JudgeBench] grades a judge on whether it picks the correct answer, where earlier judge benchmarks mostly measured agreement with human preference. It contains 350 response pairs in knowledge, reasoning, mathematics, and coding, and in each pair one response is objectively correct and the other is not. The pairs are built by converting existing hard datasets that have ground-truth labels, so the label does not depend on what a crowd of raters would have preferred. Many strong models, used as judges, did only slightly better than random guessing in the paper. That result marks the limit of a judge calibrated on preference data: on questions hard enough that the judge could not reliably answer them itself, agreement with raters says little about correctness.

Repository: <https://github.com/ScalerLab/JudgeBench>

ChatEval

ChatEval [Chan et al., ChatEval] is a panel of LLM agents that debate before reaching a judgment. It is at once a coordination setup and an evaluation method, which makes it useful when a team wants to study whether debate improves agreement with human raters.

Repository: <https://github.com/thunlp/ChatEval>

Agent-as-a-Judge

Agent-as-a-Judge [Zhuge et al., Agent-as-a-Judge] introduces judges that are themselves agents, inspecting the intermediate artifacts of the system under test along with its final output. What a judge can inspect determines what it can reason about, so an agentic judge can grade the process as well as the result.

Repository: <https://github.com/metauto-ai/agent-as-a-judge>

WebDevJudge

WebDevJudge [Li et al., WebDevJudge] is a meta-evaluation of whether LLM judges can stand in for human experts on open-ended web development, scored against expert preference labels. The best judges still trail human agreement by a wide margin, reject implementations that behave the same but read differently, and keep a position bias that instructions do not remove, which marks where a calibration loop should concentrate.

Repository: <https://github.com/lcy2723/WebDevJudge>

Embodied agents

Embodied benchmarks move evaluation past text in and text out, to agents whose actions land in a physical or simulated world.

EmbodiedBench

EmbodiedBench [Yang et al., EmbodiedBench] spans 1,128 multimodal embodied tasks across four environments, ranging from high-level semantic tasks to low-level atomic actions, with capability subsets, such as spatial awareness and long-term planning, that show where a model falls short. More than a thousand tasks also separate models with less sampling noise than a suite of a few dozen.

Repository: <https://github.com/EmbodiedBench/EmbodiedBench>

Embodied Agent Interface

Embodied Agent Interface [Li et al., Embodied Agent Interface] is a common interface for evaluating LLMs on four decision-making modules (goal interpretation, subgoal decomposition, action sequencing, and transition modeling), with metrics that separate hallucination errors, affordance errors, and several kinds of planning error. It pairs with EmbodiedBench: one measures competence at scale, the other tells you which module failed and how.

Repository: <https://github.com/embodied-agent-interface/embodied-agent-interface>

PhysBench

PhysBench [Chow et al., PhysBench] probes physical-world understanding for vision-language models, covering object properties, relationships between objects, scene understanding, and physics-based dynamics. Vision-language models that handle common-sense reasoning well still struggle with the physical world, and for embodied and robotics agents this benchmark sets the floor.

Repository: <https://github.com/physical-superintelligence-lab/PhysBench>

Reasoning and modality-specific

These narrow tests measure capabilities an agent depends on when it reasons over one modality.

MathVista

MathVista [Lu et al., MathVista] collects 6,141 problems of mathematical reasoning in visual contexts, such as diagrams, charts, geometry, and figures from papers, drawn from 28 existing datasets and three new ones. It exposes the difference between solving a math problem stated in text and solving it when the reader first has to understand a figure.

Repository: <https://github.com/lupantech/MathVista>

MMAU

MMAU [Sakshi et al., MMAU] covers multi-task audio understanding and reasoning across speech, environmental sounds, and music. It is a broad capability benchmark for agents that listen.

Repository: <https://github.com/Sakshi113/MMAU>

MC-Search

MC-Search [Ning et al., MC-Search] tests agentic search across images and text, with long reasoning chains. It tests whether an agent can ground a search in more than one modality and reason about the results, beyond retrieving them.

Repository: <https://github.com/YennNing/MC-Search>

Reflection-Bench

Reflection-Bench [Li et al., Reflection-Bench] targets epistemic agency, the ability to build, adapt, and monitor beliefs about a changing environment, measured across seven dimensions: prediction, decision-making, perception, memory, counterfactual thinking, belief updating, and meta-reflection. It is a direct probe of whether a model notices when its own picture of the world has gone stale.

Repository: <https://github.com/AI45Lab/ReflectionBench>

The catalog is deliberately broader than most systems need. A benchmark earns a place in the evaluation portfolio when it informs a named decision, exposes a relevant failure surface, and contributes something the private suite does not already measure; the rest can stay on the shelf.

